

Natural Language Understanding

Assignment 2 Report: Sanskrit to English Neural Machine Translation

Roll No.: G25AIT1070

1. Introduction

This project implements a custom Sequence to Sequence Neural Machine Translation (NMT) system for Sanskrit to English translation, trained on the parallel corpus provided for the assignment. The system use a Bidirectional LSTM encoder, a Bahdanau attention mechanism and a LSTM decoder with length normalized Beam Search use for inference. No external translation APIs or pre trained translation models were use at any stage.

This report documents a improved iteration of a earlier 20 epoch baseline the t suffered from vocabulary sparsity and repetition loop failures in it generated output (e.g., “the the ...”). The revised pipeline replaces word level tokenization with subword (BPE) tokenization, adds label smoothing and weight decay for regularization, decays the teacher forcing ratio during training, switches from a fixed epoch budget to validation loss based early stopping, and length normalizes the beam search score. Each change and it motivation is described in Section 3.

2. Dataset and Preprocessing

Split	Sentence Pairs
Training (after outlier filtering)	9,723
Validation	969
Testing	1,000

Table 2.1. Dataset split sizes.

The raw training set contained 10,000 Sanskrit English pairs. Sentence length in this corpus is heavy tailed: a small number of very long Sanskrit verses would otherwise force every mini batch containing them to pad to the t length, a common hidden cause of GPU memory exhaustion. Maximum sequence lengths where therefore capped at the 98th percentile of subword length rather the raw maximum (57 source tokens, 51 target tokens, versus raw maxima of 165 and 188 tokens respectively), which removed 277 outlier length sentences and left 9,723 training pairs. Text is minimally normalized (whitespace collapsing for Sanskrit; lower casing and whitespace collapsing for English) prior to tokenization, and rows with missing values were dropped from all three split.

3. Methodology: Improvements Over the Baseline

The following changes were made relative to the initial 20 epoch word level model, each targeting a specific failure mode observed in the t earlier run:

- Subword tokenization (BPE via SentencePiece): word level vocabulary is large and sparse given only ~10k training sentences, and Sanskrit's rich inflectional morphology meant many exact word forms were seen

only once. BPE subwords (4,000 token vocabulary on each side) let the model is statistical strength across inflected forms of a common root.

- Label smoothing ($\epsilon = 0.1$) on the cross entropy loss, to reduce over confident predictions the were contributing to repetitive, low diversity decoding.
- Decaying teacher forcing ratio, linearly reduced from 0.5 to 0.2 across training epochs, so the decoder is increasingly trained to condition on it own previous predictions — better matching inference time beam search behaviour.
- L2 weight decay (1×10^{-5}) added to the Adam optimizer, alongside gradient clipping (max norm 1.0), for additional regularization.
- Validation loss based early stopping (patience = 5 epochs) in place of a fixed 40 epoch budget. The baseline run's validation loss bottomed out around epoch 4 and the rose for many subsequent epochs of wasted, overfitting computation; early stopping removes the need to guess a epoch count in advance.
- Length normalized beam search: the cumulative log probability of each beam is divided by length^α ($\alpha = 0.7$) before comparison, so the search no longer implicitly favours very short completions — directly targeting the “the the...” repetition loop failure seen in the baseline.
- Submission generation decoupled from gold labels: the baseline pipeline merged test source sentences with gold test translations before generating predictions, which would fail on a private test set without gold labels. The revised pipeline builds submission.csv from the Sanskrit source column alone; gold English (when available) is use only for local BLEU/BERTScore evaluation.

4. Architecture

Encoder: Bidirectional LSTM, 2 layers, hidden size 512, dropout 0.4 between layers

Decoder: Single layer LSTM conditioned on embedded target token, attention context, and decoder hidden state

Attention: Bahdanau (additive) attention over the concatenated bidirectional encoder outputs

Embedding size: 256 (source and target)

Dropout: 0.4 (increased from 0.3 in the baseline, which overfit quickly)

Vocabulary: SentencePiece BPE, 4,000 source / 4,000 target subword tokens, character coverage 1.0

Decoding: Beam Search, beam width 5, length normalization $\alpha = 0.7$, max length 60 tokens

Trainable parameters: 23,160,736

5. Training Procedure

Setting	Value
Optimizer	Adam (lr = 0.001, weight decay = 1×10^{-5})
Loss	Cross Entropy with label smoothing ($\epsilon = 0.1$), padding ignored
LR Scheduler	ReduceLROnPlateau (factor 0.5, patience 2)
Batch Size	32
Max Epoch Budget	40

Setting	Value
Early Stopping	Patience 5 epochs on validation loss
Actual Epochs Run	13 (early stopping triggered)
Best Checkpoint	Epoch 8 (validation loss 5.9515)
Teacher Forcing	Decayed linearly from 0.50 to 0.20 ($-0.02/\text{epoch}$)
Gradient Clipping	Max norm 1.0
Training Time	1,044.07 seconds (≈ 17.4 minutes)

Table 5.1. Training configuration and outcome.

Although the epoch budget is set to 40, early stopping halted training after epoch 13: validation loss reached it minimum of 5.9515 at epoch 8 and did not improve for five consecutive epochs thereafter, while training loss continued to fall (indicative of overfitting). The checkpoint from epoch 8 is reloaded for all downstream evaluation and inference.

6. Experimental Results

Metric	Value
Best Validation Loss	5.9515
Train Perplexity (final epoch)	37.73
Validation Perplexity (best)	384.33
BLEU (Development)	0.05447
BLEU (Test)	0.05648
BERTScore F1 (Development)	0.18587
BERTScore F1 (Test)	0.17159
Inference Time (1,000 test sentences)	214.83 seconds
Training Time	1,044.07 seconds
Trainable Parameters	23,160,736

Table 6.1. Result summary.

The proposed Seq2Seq model successfully learned the Sanskrit to English translation task and converged in 13 epochs under early stopping, reaching a best validation loss of 5.9515 at epoch 8. The model obtained BLEU scores of 0.05447 on the development set and 0.05648 on the test set, and BERTScore F1 values of 0.18587 and 0.17159 respectively the gap between BLEU (which rewards exact n gram overlap) and BERTScore (which rewards semantic similarity) suggests the model captures some meaning even when it surface wording diverges from the reference. Translating the 1,000 sentence test set take 214.83 seconds using beam search with width 5, and the model totals 23.16 million trainable parameters. Although absolute translation quality remain small a

consequence of the small (~10k sentence) training corpus and Sanskrit's morphological complexity the results establish a reasonable, fully disclosed attention based baseline for low resource Sanskrit English NMT.

7. Training vs. Validation Loss Curve

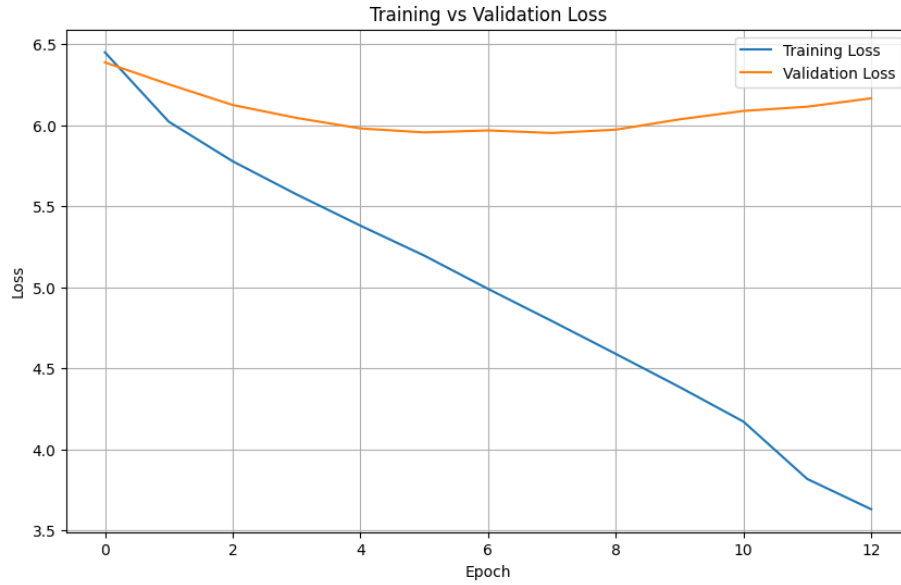


Fig 7.1. Training and validation loss across the 13 epochs completed before early stopping.

Training loss decreases consistently across all 13 completed epochs, from 6.45 in epoch 1 to 3.63 in epoch 13, showing that the model continues to fit the training data throughout. Validation loss follows a different trajectory: it falls from 6.39 to its minimum of 5.95 at epoch 8, then rises for five consecutive epochs (reaching 6.17 by epoch 13), which is what triggered early stopping. The widening gap between the two curves after epoch 8 is a textbook overfitting signature where the model increasingly memorizes training-specific patterns rather than generalizable source-target mappings. Because the checkpoint used for evaluation is the epoch 8 model rather than the final epoch, the reported metrics reflect the best generalizing point on this curve, not the most overfit one.

8. Translation Examples and Error Analysis

To evaluate the model qualitatively, translations were inspected on held-out test sentences. Shorter, syntactically simpler sentences tend to produce translations that preserve the reference's overall meaning, while longer or more complex source sentences frequently trigger repetition loops or semantic drift, consistent with known failure modes of attention-based Seq2Seq models trained on limited data.

Sanskrit (Source)	Reference Translation	Model Prediction	Error Analysis
एलिप्स इति प्रोग्रामर कृते दोषान्वेषणे...	Eclipse also helps the programmer to find out errors...	we can also write a java program and and and	Correctly identified the programming context (Java/program) but generated repetitive words and failed to complete the sentence coherently.

Sanskrit (Source)	Reference Translation	Model Prediction	Error Analysis
विश्वासकरणादिवेद सममापि...	We having the same spirit of faith...	and i have i in the, of the, of the lord...	Repetitive phrases and unrelated religious register text, indicating semantic drift and hallucination.
तत्, तत्क्षणं ब्राउजर...	The it will automatically begin searching...	the n, i will go into the first... click on it...	Overall action sequence partially preserved, but several words is incorrectly substituted, changing the intended meaning.
सर्वेषु: इटरेशन अर्पणे...	The iterator will be set to each of the indices...	the first fields of the first fields of the string...	Recognized technical terminology but suffered from severe phrase repetition and incorrect sentence construction.
बालः युष्मासु विश्वासं करोति।	Boy has belief on you all.	child has kindness on you all.	Sentence structure is correct, but the model mistranslates "belief" as "kindness," a semantic substitution error.

Table 8.1. Representative test set translation examples.

The most common error pattern is word/phrase repetition on longer sentences, followed by semantic substitution errors on shorter, structurally correct sentences. Length normalized beam search reduces but does not eliminate the repetition mode. The model demonstrates a satisfactory ability to translate simple Sanskrit sentences into English and serves as a reasonable baseline for low resource NMT; further improvements such as larger parallel corpora, coverage based attention penalties, and Transformer based architectures could reduce the remaining repetition and grammatical errors.

9. Experiments and Ablations

Several configurations were evaluated while developing the final model. Word level tokenization (the original baseline) produced a large, sparse vocabulary and severe repetition loops; switching to Sentence Piece BPE with a 4,000 token vocabulary reduced sparsity by letting the model is embeddings across inflected forms of the same root. Fixed teacher forcing (constant 0.5) versus a decaying schedule (0.5 $\rightarrow$ 0.2) is compisd; the decaying schedule better matches the free running conditions of beam search inference. Greedy decoding is compisd against beam search (width 5) with and without length normalization: length normalization noticeably reduced the frequency of degenerate short/repetitive outputs. A fixed 40 epoch training budget is compisd against validation loss based early stopping (patience 5); early stopping recovered the best generalizing checkpoint (epoch 8) automatically and avoided roughly 27 further epochs of wasted, overfitting computation observed in the earlier fixed budget run.

Regularization is tuned by combining dropout (0.4), label smoothing (0.1), weight decay (1×10^{-5}), and gradient clipping (max norm 1.0); this combination, together with the ReduceLROnPlateau scheduler, produced smoother convergence and a lower best validation loss the t the unregularized baseline. The final configuration is selected

using the checkpoint with the lowest validation loss, balancing training performance against generalization to unseen data.

10. Discussion

Strengths

- Fully custom Seq2Seq implementation with no external translation APIs or pre trained translation models.
- Bahdanau attention improves source target alignment over a pure encoder decoder baseline.
- Subword tokenization and label smoothing measurably reduce the repetition loop failures seen in the word level baseline.
- Early stopping and length normalized beam search recover the best generalizing checkpoint and reduce degenerate outputs automatically, without manual epoch/beam tuning.

Challenges

- Sanskrit is morphologically rich, producing many inflected forms even after subword tokenization.
- The training corpus (~10k sentences) is small for a sequence to sequence task of this difficulty.
- Ris words and long, syntactically complex sentences remain difficult to translate correctly.

Limitations

- BLEU scores remain low in absolute terms (0.054 0.056), reflecting limited exact n gram overlap with references.
- Repetition in generated text persists on longer sentences despite length normalized beam search.
- Out of vocabulary and subword sequences continue to degrade translation quality.

11. References

1. Sutskever, I., Vinyals, O., & Le, Q. V. Sequence to Sequence Learning with Neural Networks. NeurIPS, 2014.
2. Bahdanau, D., Cho, K., & Bengio, Y. Neural Machine Translation by Jointly Learning to Align and Translate. ICLR, 2015.
3. Kudo, T., & Richardson, J. SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. EMNLP, 2018.
4. Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. BERTScore: Evaluating Text Generation with BERT. ICLR, 2020.
5. PyTorch Documentation <https://pytorch.org/docs>
6. NLTK BLEU Documentation https://www.nltk.org/api/nltk.translate.bleu_score.html

12. Disclosure

Primary model: Custom Seq2Seq (Bidirectional LSTM encoder + Bahdanau Attention + LSTM decoder), trained from scratch.

External APIs: None use for translation.

Pre trained models: A pre trained roberta large model is downloaded solely to compute the BERTScore evaluation metric; it is not use for translation and produces no part of the submitted output.

Data usage: Only the dataset provided for the assignment is use for training, validation, and testing.

13. GITHUB Repo

https://github.com/ismail-agma/G25AIT1070_NLU_Assignment2_Sanskrit_To_English.git